

Addressable RGB LED Strip, 5V (30, 60 or 144 LEDs per meter) (SK9822)

USER GUIDE

Using the LED strip



The connectors and power wires for our SK9822 and APA102C LED strips. On the left is the input end of the strip and on the right is the output end.

Each LED strip has three connection points: the input connector, the auxiliary power wires, and the output connector. These can be seen in the adjacent picture, from left to right: auxiliary power wires, input connector, and output connector. The strip uses 4-pin JST SM connectors.

The **input connector** has three male pins inside of a plastic connector shroud, each separated by about 0.1". The black wire is ground, the green wire is the signal input, and the red wire is the power line.

The **auxiliary power wires** are connected to the input side of the LED strip and consist of stripped black and red wires. The black wire is ground, and the red wire is the power line. This provides an alternate (and possibly more convenient) connection point for LED strip power.

The **output connector** is on the other end of the strip and is designed to mate with the input connector of another LED strip to allow LED strips to be chained. The black wire is ground, the green wire is the signal output, and the red wire is the power line.

All three black ground wires are electrically connected, and all three red power wires are electrically connected.



www.pololu.com

A close-up of the JST SM connectors for our SK9822 and APA102C LED strips.

Included hardware

These LED strips ship with flexible silicone brackets and screws. Strips with lengths of 1 meter or greater include five brackets and ten screws *per meter*. Our 0.5 meter high-density strip ships with a total of two brackets and four screws. The brackets fit over the waterproof sheath and can be used to mount the LED strip. The LED strip also ships on a plastic reel.



The 1m, 2m, and 5m addressable LED strips include five mounting brackets per meter; the 0.5m strip includes 2 total brackets.



Controlling an APA102C or SK9822 addressable RGB LED strip with an A-Star 32U4 Prime SV and powering it from a 5V wall power adapter.

Connecting the LED strip

To control the LED strip from a microcontroller, two wires from the input connector should be connected to your microcontroller. The LED strip's ground (black) should be connected to ground on the microcontroller, and the LED strip's signal input line (green) and clock input line (yellow) should be connected to one of the microcontroller's I/O lines. The male pins inside the input connector fit the female terminations on our [premium jumper wires](#) and [wires with pre-crimped terminals](#). If you are connecting the LED strip to a breadboard or a typical Arduino with female headers, you would want to use male-female wires.

We generally recommend powering the LED strip using the auxiliary power wires. Our 5 V wall power adapters work well for powering these LED strips and a DC Barrel

Jack to 2-Pin Terminal Block Adapter can help you make the connection between the adapter and the strip. However, you might need a [wire stripper](#) to strip off some more insulation from the power wires.

It is convenient that the power wires are duplicated on the input side because you can connect the auxiliary power wires to your 5 V power supply and then the power will be available on the data input connector and can be used to power the microcontroller that is controlling the LED strip. This means you can power the microcontroller and LED strip from a single supply without having to make branching power connections.

Making a custom cable

If you do not want to use our premium jumper wires to connect to the LED strip's input, it is possible to make a custom cable.

One option for making a custom cable is to cut off the unused output connector on the last LED strip in your chain. This can then be plugged into the input connector of the first LED strip. The wires on the output and input connectors are 20 AWG, which is too thick to easily use with our crimp pins and housings, but you could solder the wires to header pins.

Alternatively, you can get your own JST SM connectors and make a custom cable using those. The parts you would need to get are the SMP-04V-BC and the SHF-001T-0.8BS, which are described in the [SM Connector datasheet](#) from JST. These can be purchased from several places such as [Heilind](#). You will also need some 22–28 AWG stranded wire and a wire stripper. We do not know of a great way to crimp wires onto the JST crimp pins, but we were able to successfully do it using our narrower crimping tool and pliers. (With the wider crimping tool, it is hard to avoid crimping parts of the pin that should not be crimped.) Before crimping, use pliers to bend the outer set of tabs a little bit so that they can hold on to the insulation of the wire. This makes it easier to position the crimp pin and the wire. Next, you should be able to follow the

instructions on the crimping tool product page to crimp the wire. After that, you will probably need to squeeze the crimp pin with pliers to get it to fit into the JST plug housing. On the other end of the cable you could make a custom connector using our [crimp pins](#) and [crimp connector housings](#), which will allow you to plug it directly into a breadboard or 0.1" header pins.

Current draw

Each SK9822 LED draws approximately 50 mA when it is set to full brightness. This means that for every 30 LEDs you turn on, your LED strip could be drawing as much as 1.5 A. Be sure to select a power source that can handle your strip's current requirements.

The SK9822 has built-in constant current control. For any given color command, the SK6812's actual color and brightness are largely independent of its supply voltage as long as the voltage is between 3.5 V and 5 V. This means that voltage drops caused by the resistance in long power connections are less likely to affect the color or brightness of the light emitted.

Color

The hue of the light emitted by the SK9822 becomes less red and more green as the 5-bit brightness value is lowered. Also, the SK9822 has some unit-to-unit variation in the brightness of its red channel which can be noticed when the 5-bit brightness value is low but is usually hard to see when the 5-bit brightness value is high.

Chaining

Multiple LED strips can be chained together by connecting input connectors to output connectors. When strips are chained this way, they can be controlled and powered as one continuous strip. Please note, however, that as chains get longer, they will require

more power. If this becomes an issue, you can chain the data lines while separately powering shorter subsections of the chain.

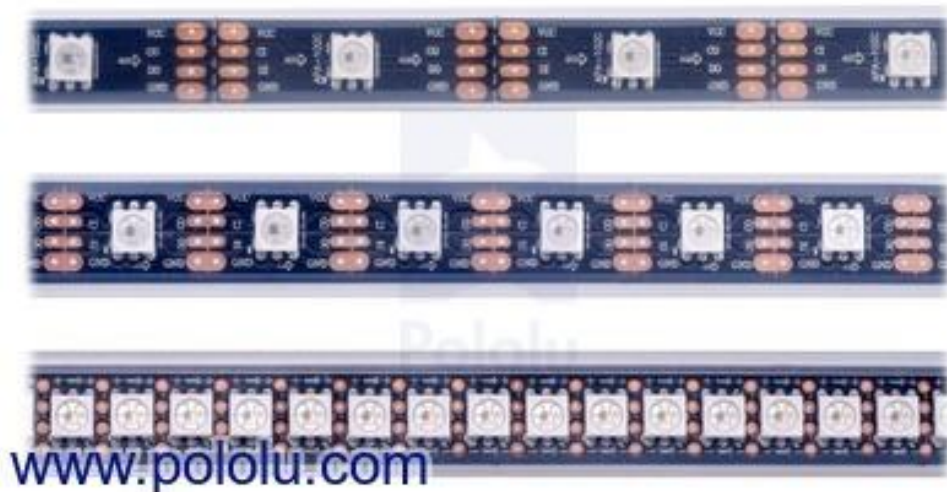


Two addressable SK9822 or APA102C RGB LED strips connected.

We recommend chains of LEDs powered from a single supply not exceed 240 total RGB LEDs. It is fine to make longer chains with connected data lines, but you should power each 240-LED section separately. If you are powering each section from a different power supply, you should cut the power wires between the sections so you do not short the output of two different power supplies together.

Cutting

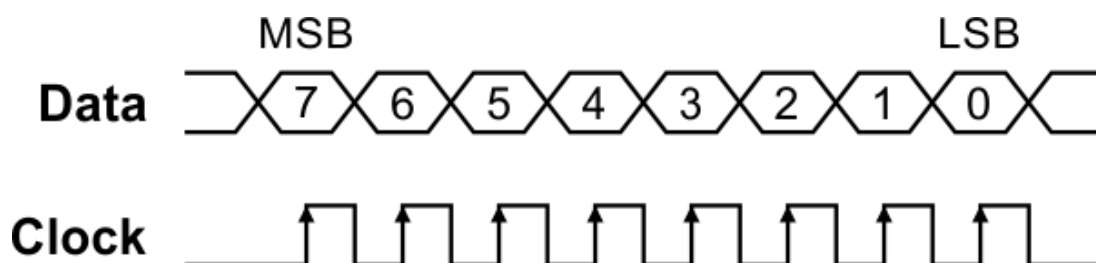
The LED strip is divided into segments, with each segment containing one RGB LED. The strip can be cut apart on the lines between each segment to separate it into usable shorter sections. The data connection is labeled **DO**, **SDO**, **DI**, or **SDI**, the clock connection is labeled **CO**, **CLKO**, **CI**, or **CLKI**, the positive power connection is labeled **VCC** or **5V**, and the ground connection is labeled **GND**. Each LED in the picture below is at the center of its own segment; there are lines printed on the PCB silkscreen where the segments can be cut.



Protocol

These LED strips are controlled through an SPI protocol on the data and clock input lines. The protocol is documented in the [SK9822 datasheet](#) (476k pdf), but we describe it below with some modifications that we have found to work better.

The default, idle state of the clock signal line is low, and the data signal is read on each rising edge of the clock. To update the LED colors, you need to toggle the clock line while driving the data line with the value of each bit to send; this can be done through software (bit-banging), or it can be handled by a hardware SPI peripheral in a microcontroller. There is no minimum clock frequency, although using a lower frequency means that it will take longer to update the entire sequence of LEDs, so you will probably want to use the fastest practical clock speed to get the best update rate.

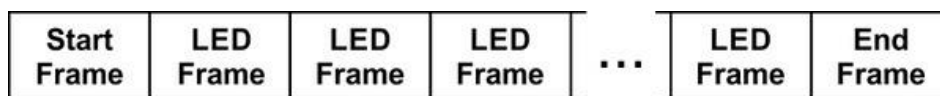


Control signal timing diagram for the SK9822 and APA102C

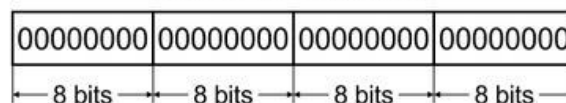
The data for each LED is encoded as a sequence of 32 bits (4 bytes) called an LED frame. The first three bits of the LED frame should be '1'. The next 5 bits are a "global", color-independent brightness value (0–31) that is applied equally to all three color channels. The remaining 24 bits are the color values, in BGR (blue-green-red) order. Each color value uses 8 bits (0–255). The most significant bit of each value is transmitted first.

To update all the LEDs in the panel, you should send a "start frame" of 32 '0' bits, then a 32-bit "LED frame" for each LED, and finally an "end frame".

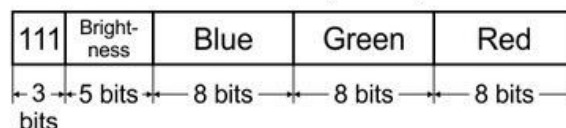
The SK9822 datasheet recommends that the end frame be composed of 32 '1' bits, but we have found this does not actually ensure that all the LEDs will start displaying their new colors immediately. This issue can be avoided by using an end frame that consists of one byte (8 bits) that are all '1', followed by $5+n/165+n/16$ bytes (rounded down to the next whole number) that are all '0'. For a more detailed explanation, see the comments in the source code of our APA102 Arduino library, discussed below.



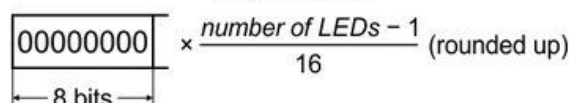
Start Frame (32 bits)



LED Frame (32 bits)



End Frame



Data format for the SK9822 and APA102C.

If you send fewer LED frames than the number of LEDs on the strip, then some LEDs near the end of the strip will not be updated, and the LED after the the last LED that you sent a color to will change to black.

For example, to update all 30 LEDs on a 1-meter strip, you should send a 4-byte start frame, thirty 4-byte LED frames, and a 7-byte end frame, for a total of 131 bytes. If multiple strips are chained together with their data connectors, they can be treated as one longer strip and updated the same way (two chained 1-meter strips behave the same as one 2-meter strip).

Each RGB LED receives data on its data input line and passes data on to the next LED using its data output line. The update rate is generally limited only by the speed of the controller; our Arduino library below can update 60 LEDs in about 1.45 milliseconds, so it is possible to update nearly 700 LEDs at 60 Hz. However, constant updates are not necessary; the LED strip can hold its state indefinitely as long as power remains connected.

Note: The minimum logic high threshold for the data and clock signals is not specified in the SK9822 datasheet, so you should consider using level shifters if you want to control these strips from 3.3 V systems. In our tests, we were able to control the LEDs directly with 3.3 V signals, but that is not guaranteed to always work.

Comparison with SK6812 LED Strips

Like the SK9822, the SK6812 used in some of our LED strips also combines an RGB LED and driver into a single 5050-size package. However, while the SK6812 uses a one-wire control interface with strict timing requirements, the SK9822 uses a standard SPI interface, with separate data and clock signals that lets it accept a wide range of communication rates; the trade-off is that two I/O lines are required to control it instead of just one.

The SK9822 provides a 5-bit brightness control that is not available on the SK6812. This feature can be used to dim the LED strip while still retaining 24-bit color control.

For more information about the ICs, see the [SK6812 datasheet](#) (459k pdf) and [SK9822 datasheet](#) (1MB pdf).

While our SK6812 strips and SK9822 strips are physically very similar, they are **not** functionally compatible with each other. The easiest way to tell them apart is to look at the strips' end connectors and the connections between each LED segment: SK6812 strips have three connections (power, data, and ground), while SK9822 strips have four (power, clock, data, and ground).

Comparison with APA102C LED Strips

The SK9822 is very similar to the APA102C used in some of our older LED products, and can be used as a drop-in replacement in many applications. LED products based on the SK9822 and APA102C can be chained together.

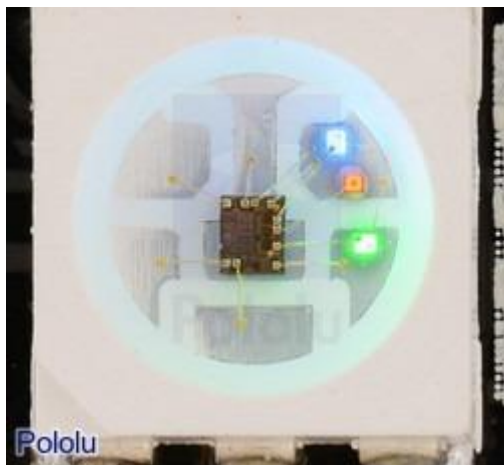
The SK9822 and APA102C ICs have different conditions for when to update the color that is being shown. The SK9822 basically waits until all the LEDs in the chain have received their colors, while the APA102C will start displaying its new color almost immediately after receiving it. The SK9822's behavior is better because it makes it easier to update a large number of LEDs without having perceptible latency. However, if you replace APA102C LEDs with SK9822 LEDs, then, depending on what type of signal you are sending, you might notice that the SK9822 LEDs are late to update. The solution is to make sure that you are sending the correct "end frame" as described above in the "Protocol" section.

The SK9822 has voltage-independent brightness over a wide voltage range, as described in the "Current draw" section, which means that the colors of the LEDs

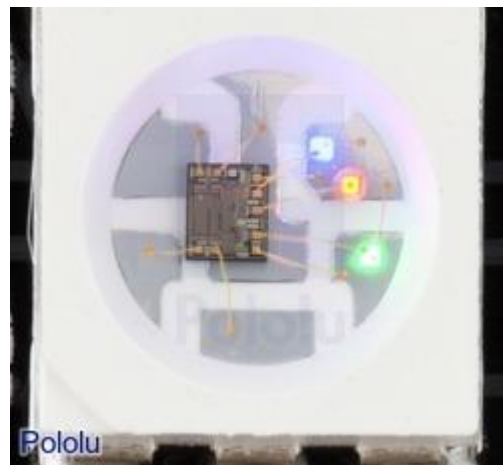
should not be affected by a drop in the supply voltage as much as they are on the APA102C.

The colors generated by the SK9822 look different from the colors generated by the APA102C. In particular, the SK9822's red channel is relatively less bright, which makes it have a blue-green tint when compared to the APA102C. While the APA102C's brightness register allows you to control the brightness of each LED independently of its color, we have noticed that the colors from the SK9822 become less red and more green as the 5-bit brightness value is lowered. Also, the SK9822 has some unit-to-unit variation in the brightness of its red channel, which can be noticed when the 5-bit brightness value is low but is usually hard to see when the 5-bit brightness value is high.

The SK9822 can be distinguished from the APA102C by visual inspection. The pictures below show what each IC looks like:



Close up of an SK9822, with the red, green, and blue LEDs on at a low brightness.



Close up of an APA102C, with the red, green, and blue LEDs on at a low brightness.